Konzeptdokument – RideLog

Projektname: RideLog – Mitfahr-Planer (Schule / Arbeit)
Name: Felix Wimplinger
Datum: 13.02.2026

1. Projektidee

RideLog ist eine Android-App zur Organisation von Mitfahrgelegenheiten im Schul- und Arbeitsalltag.

Im Alltag entstehen häufig unübersichtliche Situationen bei Fahrgemeinschaften:

- Wer fährt?
- Wie viele Plätze sind noch frei?
- Wer ist angemeldet?
- Wer hat abgesagt?

RideLog strukturiert diese Abläufe digital. Fahrer können Fahrten erstellen, Mitfahrer können Fahrten beitreten oder verlassen. Zusätzlich bietet die App eine Historie sowie statistische Auswertungen über Nutzung und Einsparungen.

Die App wird mit **Flutter (Frontend)** und einem **Express-Backend (REST API)** umgesetzt.
Die Daten werden in einer relationalen Datenbank gespeichert.

3. Funktionsumfang

- Fahrt erstellen
- Fahrt bearbeiten oder löschen (eigene Fahrt)
- Fahrt beitreten
- Fahrt verlassen
- Anzeige freier Plätze
- Historie vergangener Fahrten
- Statistische Auswertung (KM, CO_2, Anzahl Fahrten)
- Theme-Wechsel
- Profilverwaltung
- Gruppenverwaltung

4. Screens & UI-Konzept

4.1 Home – Ride Dashboard

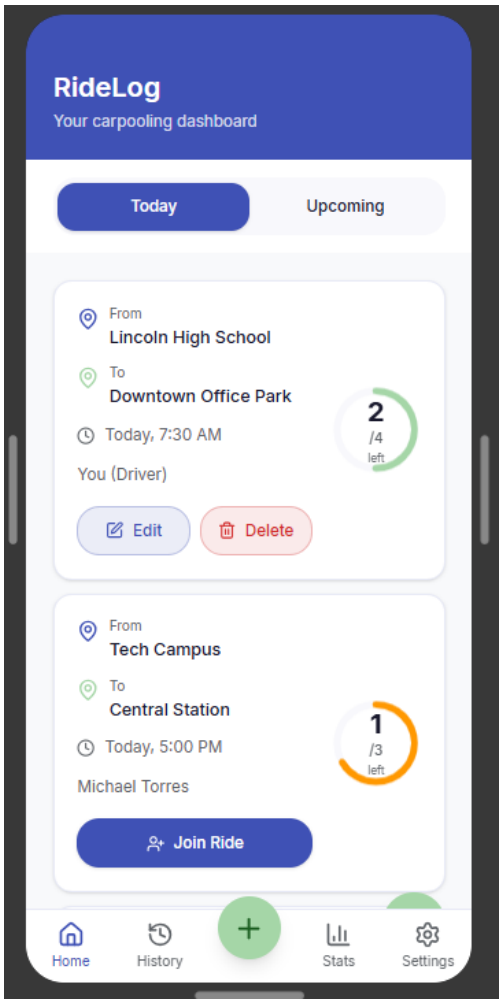
Zentrale Übersicht über aktuelle und zukünftige Fahrten.

Inhalte:

- Tab-Auswahl: Today / Upcoming
- Liste von Ride-Cards
- Anzeige:
    - Start- und Zielort
    - Datum und Uhrzeit
    - Fahrer (z.B. „You (Driver)“)
    - Kreisdiagramm für freie Plätze
- Kontextabhängige Aktionen:
    - Edit / Delete bei eigener Fahrt
    - Join Ride bei fremder Fahrt

Navigation erfolgt über eine Bottom Navigation mit:

- Home
- History
- + (für Create Ride)
- Stats
- Settings



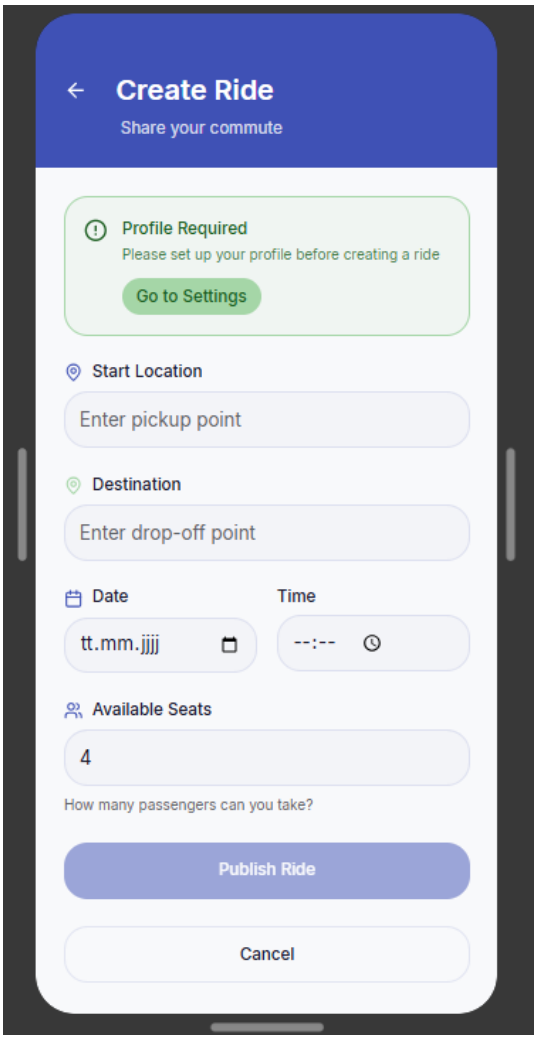
4.2 Create Ride

Ermöglicht das Erstellen einer neuen Fahrt.

Eingabefelder:

- Start Location
- Destination
- Date
- Time
- Available Seats
- Optional: Preis pro Platz
- Optional: Notiz

Es erfolgt eine Validierung der Pflichtfelder.
Nach erfolgreicher Erstellung wird die Fahrt in der Liste angezeigt.



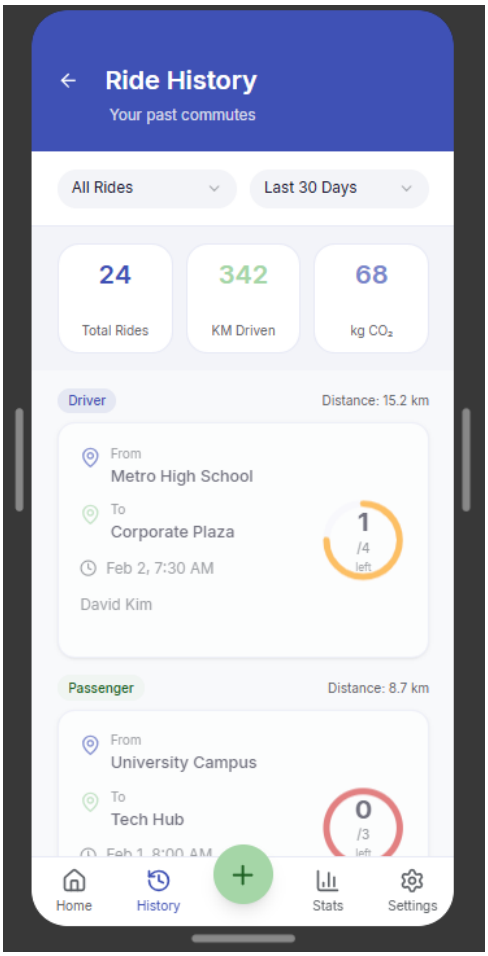
4.3 Ride History

Zeigt vergangene Fahrten.

Funktionen:

- Filter nach Zeitraum (z.B. Last 30 Days)
- Filter nach Rolle (Driver / Passenger)
- Anzeige:
    - Strecke
    - Datum
    - Gefahrene Distanz
- Zusammenfassung:
    - Total Rides
    - KM Driven

- kg CO_2



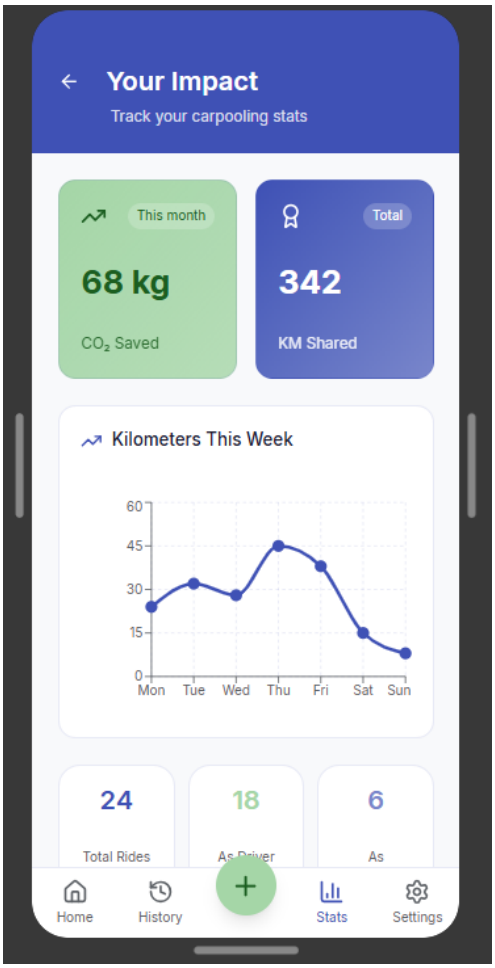
4.4 Stats – Your Impact

Statistische Auswertung der Nutzung.

Anzeige:

- CO_2 Saved
- KM Shared
- Anzahl Fahrten
- Aufteilung:
    - Als Fahrer
    - Als Mitfahrer
- Wochen-Diagramm der gefahrenen Kilometer

Die Daten werden aus der Datenbank aggregiert.



4.5 Settings

Enthält folgende Bereiche:

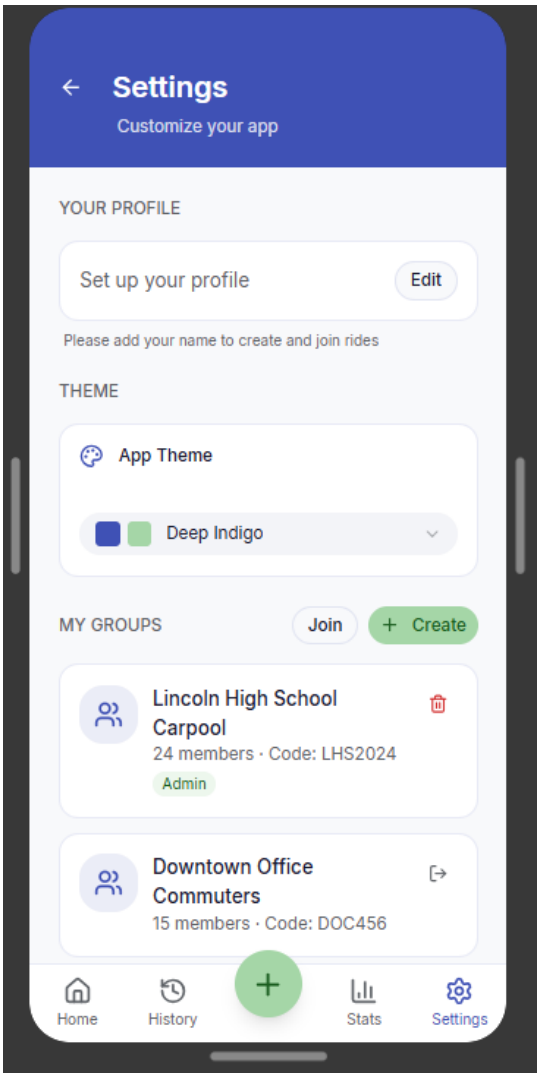
Profil:

- Name bearbeiten
- Profil einrichten

Themesauswahl

Gruppen:

- Gruppenübersicht
- Mitgliederanzahl
- Gruppen-Code
- Join- oder Create-Funktion



5. Programmablauf

5.1 Fahrt erstellen

Benutzer klickt auf den Plus-Button.

App prüft, ob ein Profil (Benutzername) eingerichtet ist.

Falls kein Profil vorhanden ist, erscheint ein Hinweis („Profile Required“) und Weiterleitung zu den Settings.

Formular wird ausgefüllt (Start, Ziel, Datum, Uhrzeit, Plätze, optional Preis).

Frontend überprüft die Eingaben (Pflichtfelder, gültige Werte).

Daten werden per POST-Request an das Backend gesendet.

Backend speichert die Fahrt in der Tabelle rides.

Erfolgsbestätigung wird zurückgegeben.

Home-Screen aktualisiert sich automatisch und zeigt die neue Fahrt an.

5.2 Fahrt beitreten

Benutzer öffnet eine Fahrt in der Detailansicht.

Klick auf „Join Ride“.

Backend speichert einen Eintrag in ride_passengers mit Status „joined“.

Anzahl freier Plätze wird neu berechnet.

Status ändert sich in der UI zu „Angemeldet“.

Falls keine Plätze mehr frei sind, wird eine entsprechende Fehlermeldung angezeigt.

5.3 Fahrt verlassen

Benutzer klickt auf „Cancel“.

Status in ride_passengers wird auf „cancelled“ gesetzt.

Plätze werden wieder freigegeben.

UI aktualisiert sich und zeigt den neuen Status an.

5.4 Fahrt bearbeiten oder löschen

Nur möglich, wenn der Benutzer der Fahrer ist (driver_user_id entspricht der aktuellen User-ID).

Bearbeiten:

Formular wird mit bestehenden Daten geladen.

Änderungen werden gespeichert.

Backend aktualisiert den Eintrag in rides.

UI wird aktualisiert.

Löschen:

Fahrt wird aus der Datenbank entfernt.

Zugehörige Einträge in ride_passengers werden ebenfalls gelöscht.

Die Fahrt verschwindet aus der Liste.

5.5 Benutzerprofil verwalten

Benutzer öffnet den Settings-Screen.

Unter „Your Profile“ kann der Name eingegeben oder geändert werden.

Der Name wird lokal gespeichert und optional im Backend aktualisiert.

Der Benutzername wird bei:

erstellten Fahrten (als Fahrer)

Mitfahrten (Teilnehmerliste)

angezeigt.

Ohne eingerichtetes Profil können keine Fahrten erstellt oder betreten werden.

5.6 Gruppen verwalten

Benutzer öffnet den Settings-Screen.

Unter „My Groups“ werden bestehende Gruppen angezeigt.

Benutzer kann:

einer Gruppe per Code beitreten („Join“)

eine neue Gruppe erstellen („Create“)

Beim Beitritt:

Gruppen-Code wird überprüft.

Benutzer wird der Gruppe zugeordnet.

Beim Erstellen:

Neue Gruppe wird im Backend gespeichert.

5.7 Historie & Statistik

Beim Öffnen der Statistik-Seite:

App sendet GET-Request an /stats/rides.

Backend aggregiert:

Anzahl der Fahrten

Summe der gefahrenen Kilometer

CO_2-Einsparung (basierend auf Kilometerwerten)

Aufteilung nach Rolle (Driver / Passenger)

JSON-Daten werden zurückgegeben.

5.8 Theme-Wechsel

Benutzer wählt im Settings-Screen ein Theme (System / Hell / Dunkel / weitere Varianten).

Auswahl wird lokal gespeichert.

App aktualisiert die ThemeData.

UI wird neu gerendert.

Theme bleibt auch nach Neustart der App erhalten.

6. Datenbankdesign

users

- id (PK)
- name
- email
- created_at

rides

- id (PK)
- driver_user_id
- start_name
- end_name
- depart_time
- seats_total
- price_per_seat
- distance_km
- note
- created_at

ride_passengers

- ride_id
- user_id
- status (joined / cancelled)
- joined_at

7. Backend (Express REST API)

- POST /rides
- GET /rides
- GET /rides/:id
- POST /rides/:id/join
- POST /rides/:id/cancel
- GET /stats/rides

8. Technische Umsetzung

Frontend

- Flutter
- Provider (State Management)
- fl_chart (Diagramme) (Zusätzliches, neues Konzept)
- Animations (Zusätzliches, neues Konzept)
- Eventuell Google Maps Integrierung

Backend

- Node.js
- Express
- MySQL oder SQLite

9. Meilenstein-Plan

Bis 13.02.2026

- Mockups erstellen
- Konzeptdokument erstellen

Bis 24.02.2026

- Datenbank implementieren
- API-Endpunkte erstellen
- Backend-Grundstruktur fertigstellen

Bis 09.03.2026

- Fahrten erstellen und anzeigen
- Fahrhistorie anzeigen
- Join / Cancel Funktion

Bis 13.03.2026

- Statistik implementieren
- Theme-Wechsel / Settings Seite implementieren